

Smart Expense Tracker

An object-oriented programming solution for efficient personal finance management. This system transforms manual, error-prone expense tracking into an automated, reliable process for students and individuals managing their finances.



Why Build This System?



Manual Tracking Issues

Paper receipts and spreadsheets lead to human errors, forgotten transactions, and time-consuming calculations



System Solution

Automated tracking with data validation, instant calculations, and organized storage eliminates manual errors

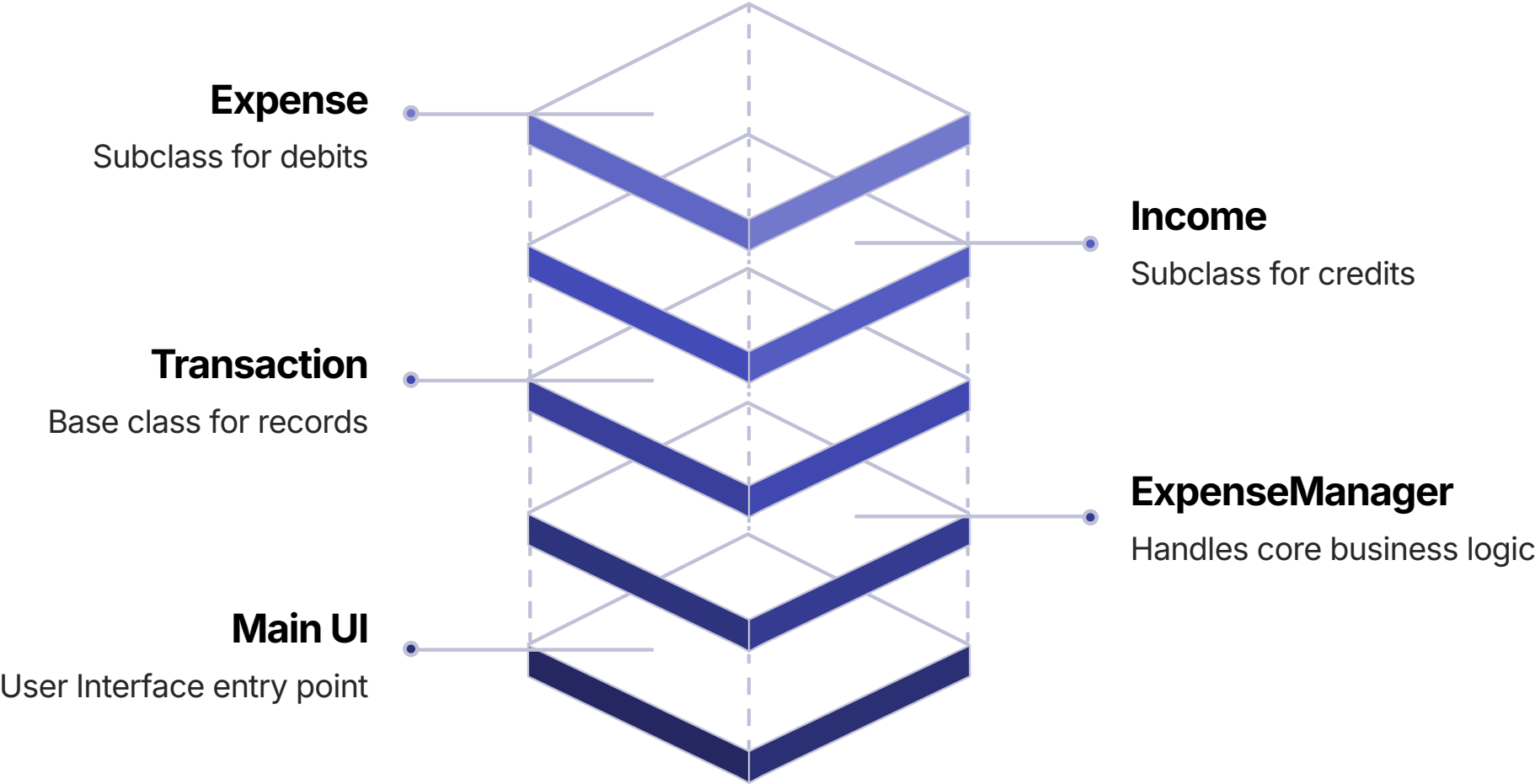


Perfect For

Students learning programming whilst managing tight budgets and individuals seeking better financial control

System Architecture Overview

The expense tracker follows a clear hierarchical structure, with each component playing a specific role in managing financial data.



Core Class Structure



Transaction

Abstract base class defining common properties: amount, category, and date with encapsulated private variables



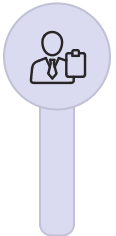
Income

Child class extending Transaction with specific display format for income sources like salary or gifts



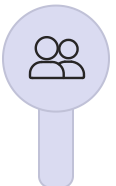
Expense

Child class extending Transaction with specific display format for spending categories like food or transport



ExpenseManager

Central logic handler managing transaction list, calculations, and search functionality with comprehensive methods



Main

User interface handling input/output operations and menu navigation for seamless user interaction

Object-Oriented Programming Principles

This implementation demonstrates four fundamental OOP concepts working together to create a robust, maintainable system.

1

Abstraction

The Transaction class defines a common interface whilst hiding implementation details. Users interact with a unified system without knowing internal complexities.

2

Inheritance

Income and Expense classes inherit from Transaction, reusing common attributes whilst adding specific functionality. This promotes code reuse and reduces redundancy.

3

Polymorphism

The display() method behaves differently based on the object type. Calling display() on Income shows income format whilst Expense shows spending format automatically.

4

Encapsulation

Private variables prevent direct access, with public getter methods providing controlled access. This protects data integrity and prevents unauthorized modifications.

Key Technical Implementation

The Polymorphic Loop

The core functionality relies on a simple yet powerful loop that demonstrates runtime polymorphism:

```
for (Transaction t :  
    transactions) {  
    t.display();  
}
```

This single loop handles both Income and Expense objects correctly, calling the appropriate display method for each type.

Supports Multiple Types

Processes Income and Expense objects in a single collection without type checking

Clean Design

Minimal code with maximum functionality through polymorphic behavior

Extensible

New transaction types can be added without modifying existing loop structure

System Capabilities



Record Income

Add income transactions with amount, source, and date. Automatically updates running balance and categorizes earnings for future reference.



Track Expenses

Log spending with detailed categories, amounts, and descriptions. System validates input and prevents negative values or empty categories.



View Transactions

Display all recorded transactions in chronological order with formatted output showing type, amount, category, and date for easy reference.



Calculate Summary

Instantly compute total income, total expenses, and current balance. Provides quick financial health overview with a single command.



Search by Category

Filter transactions by specific categories like Food, Transport, or Entertainment. Helps identify spending patterns and budget allocation.



Input Validation

Gracefully handles invalid inputs with clear error messages. Prevents system crashes and ensures data integrity through robust exception handling.



Live Demo: Adding Transactions

01

Launch Application

Start the program and view the main menu with available options

02

Add Income

Select option to add income, enter amount (5000) and category (Salary)

03

Record Expense

Choose to add expense, input amount (2000) and category (Food)

04

Verify Entry

System confirms transaction added successfully with summary details

Live Demo: Viewing & Searching



View All

Display complete transaction history showing all income and expenses with formatted details



Summary

Calculate and show total income (5000), total expenses (2000), and balance (3000)



Search

Find all Food category transactions, displaying matching expenses with amounts

Key Takeaways

Practical Application

Real-world solution to common financial management challenges using object-oriented principles effectively

Technical Excellence

Polymorphic design patterns, inheritance hierarchy, and encapsulation create maintainable, extensible code

Learning Value

Perfect project for understanding OOP concepts through hands-on implementation with immediate practical benefits

This expense tracker demonstrates how fundamental programming concepts solve real problems whilst building valuable technical skills for future software development.